

$$a^n b^m : n > m$$

$$S \rightarrow aSb \mid X$$

$$X \rightarrow a \mid Xa$$

$$a^n b^m a^n : n, m \geq 0$$

$$S \rightarrow aSa \mid B$$

$$B \rightarrow \varepsilon \mid bB$$

$$a^i b^j c^k: i+j=k$$

$$S \rightarrow \varepsilon \mid aSc \mid X$$

$$X \rightarrow bXc \mid \varepsilon$$

$$a^i b^{i+k} c^k : i+k=j$$

$$\underline{a^i} \underline{b^i} \underline{b^k} \underline{c^k}$$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid \epsilon$$

$$B \rightarrow bBc \mid \epsilon$$

$a^n b^m c^p: p \geq m$
or $m = p$

$S \rightarrow A_{nzm} \mid A_{m=p}$

$A_{nzm} \rightarrow A_{nzm}c \mid X_1$

$X_1 \rightarrow aX_1b \mid \epsilon \mid aX_1$

$A_{m=p} \rightarrow aA_{m=p} \mid X_2$

$X_2 \rightarrow bX_2c \mid \epsilon$

$$a^i b^j c^k: \bar{i} = j$$

$$\text{or } \bar{j} = k$$

$$S \rightarrow A_{\bar{i}=j} \mid A_{\bar{j}=k}$$

$$A_{\bar{i}=j} \rightarrow A_{\bar{i}=j} c \mid X_1$$

$$X_1 \rightarrow a X_1 b \mid \epsilon$$

$$A_{\bar{j}=k} \rightarrow a A_{\bar{j}=k} \mid X_2$$

$$X_2 \rightarrow b X_2 c \mid \epsilon$$

Equal # of a's, b's

babbabaa

#a > #b

#a < #b

-----a-----b-----

-----b-----a-----

Equal # of a's, b's

babbabaa

$S \rightarrow \epsilon \mid aSbS \mid bSaS$

a...b...

5) $L = \{ w \in \{0,1\}^* \mid |w| \text{ is odd and the middle symbol is } 0 \}$

(strings whose length is odd and whose central character is 0; sides can be arbitrary but must have equal length)

CFG:

nginx

 Copy code

```
S → 0
S → 0 S 0
S → 0 S 1
S → 1 S 0
S → 1 S 1
```

Why: S generates strings with a central 0 and equal-length left/right parts by adding one symbol on each side (left symbol and right symbol can be chosen independently).

Examples: 0 (length 1), 010, 10001, 11011, 0110?  note: 0110 is even length so not in L.

7) Balanced parentheses over $\Sigma = \{ (,) \}$ (the classical Dyck language)

CFG:

SCSS

 Copy code

```
S → λ  
S → S S  
S → ( S )
```

Why: λ base; (S) makes a matched pair; $S S$ concatenates balanced pieces.

Examples: λ , $()$, $((()))$, $()()$, $((())())$, etc.

8) $L = \{ 0^n 1^m \mid 2n \leq m \leq 3n \}$

CFG:

nginx

 Copy code

$S \rightarrow \lambda$

$S \rightarrow 0 S 11$

$S \rightarrow 0 S 111$

Why: each 0 contributes either 11 (2 ones) or 111 (3 ones); hence for n zeros you get between $2n$ and $3n$ ones.

Examples: λ ($n=0, m=0$), $0 11$ ($n=1, m=2$), $00 11111$ ($n=2, m=5$).

(If you need to require $n \geq 1$ remove the $S \rightarrow \lambda$ and adjust accordingly.)

more a's than b's

abaa b

$S \rightarrow E a E \mid SS$

$E \rightarrow \epsilon \mid a E b E \mid b E a E$

11) $T = \Sigma = \{0, 1, \cup, *, (,), \lambda\}$, $L = \{\text{all valid regular expressions over } \{0, 1\}\}$

Grammar for syntactically valid regular expressions (standard):

Let R be the start nonterminal (regex):

SCSS

 Copy code

```
R → R + R      (union; use `+` or `U`)  
R → R R        (concatenation; adjacency)  
R → R *        (Kleene star)  
R → ( R )      (parenthesized expression)  
R → 0  
R → 1  
R → λ
```

Notes / priorities:

- This grammar is ambiguous (usual for RE syntax). In implementations you often refine using precedence (star highest, concatenation next, union lowest).
- To derive $(0 \cup (01))$ (your example):

$R \Rightarrow (R) \Rightarrow (R + R) \Rightarrow (0 + R) \Rightarrow (0 + \downarrow R) \Rightarrow (0 + 0 1)$ — the exact derivation depends on how you parse concatenation, but the grammar above admits that expression

$\overline{a^n b^n}$

ba

$\#a=i$

$\#b=j$

$i < j$

OR $j < i$

$S \rightarrow A | B_{i < j} | B_{i > j}$

$A \rightarrow aA | bA | Aa | Ab | ba$

$B_{i < j} \rightarrow aB_{i < j}b | X_b$

$X_b \rightarrow b | X_bb$

$B_{i > j} \rightarrow aB_{i > j}b | X_a$

$X_a \rightarrow a | X_aa$

$$\overline{a^n b^n c^n} \quad a^L b^J c^K: \begin{array}{l} i \neq j \\ \text{or } j \neq k \\ \text{or } L \neq K \end{array}$$

$$X_{ba} \rightarrow aX_{ba} \mid bX_{ba} \mid cX_{ba} \mid X_{ba}a \mid X_{ba}b \mid X_{ba}c \mid ba$$

$$X_{ca} \rightarrow - - - -$$

$$X_{cb} \rightarrow - - - -$$

$$A_{i \neq j} \rightarrow A_{i > j} \mid A_{i < j}$$

$$A_{i > j} \rightarrow A_{i > j}c \mid A_{i > j,2}$$

$$A_{i > j,2} \rightarrow aA_{i > j,2}b \mid X_a$$

$$(27) \quad L = \{ (ab)^n (cb^{m_n})^n \mid n \geq 0, m_n > 0 \}$$

- Productions:

- $S \rightarrow \epsilon$
- $S \rightarrow ab S X$
- $X \rightarrow c B$
- $B \rightarrow b B \mid b$

Explanation:

- S generates the overall string. The production $S \rightarrow \epsilon$ handles the case where $n = 0$.
- The production $S \rightarrow ab S X$ adds one "ab" on the left and one "X" on the right, ensuring that the number of "ab" sequences matches the number of "X" sequences (which correspond to "c" followed by b's).
- $X \rightarrow c B$ generates a "c" followed by a sequence of one or more b's via B .
- $B \rightarrow b B \mid b$ generates one or more b's.

This CFG allows m to vary for each "cb^m" sequence in a string, as B can generate different numbers of b's for each X .

(20) $L = \{ w \in \{0,1\}^* \mid \text{each block of '0's' is followed by} \\ \text{at least the same number of '1's'.} \}$

The language L consists of all strings over the alphabet $\{0, 1\}$ where every block of zeros is followed by at least the same number of ones. This means that for every contiguous sequence of k zeros, there must be a contiguous sequence of at least k ones immediately following it. The string may have leading ones or trailing ones, and multiple blocks of zeros and ones are allowed.

A context-free grammar (CFG) for L is defined as follows:

- **Start symbol:** S
- **Productions:**
 - $S \rightarrow US \mid \epsilon$
 - $U \rightarrow 0U1 \mid U1 \mid \epsilon$

Explanation:

- The non-terminal S generates the entire string. It can be a sequence of U strings or the empty string.
- The non-terminal U generates strings that consist of a block of zeros followed by a block of ones, where the number of ones is at least the number of zeros. Specifically:
 - $U \rightarrow 0U1$ ensures that for each zero, a one is added, maintaining balance.
 - $U \rightarrow U1$ allows extra ones to be added, ensuring that the number of ones can exceed the number of zeros.
 - $U \rightarrow \epsilon$ allows the generation of empty strings or strings consisting only of ones.

This grammar correctly generates all strings in L , including those with leading ones, trailing ones, and multiple blocks of zeros and ones. For example, the string "0011011" can be derived as follows:

- $S \rightarrow US \rightarrow U(US) \rightarrow UU\epsilon$
- First U derives "0011" via $U \rightarrow 0U1 \rightarrow 0(0U1)1 \rightarrow 00\epsilon11 = 0011$.
- Second U derives "011" via $U \rightarrow 0U1 \rightarrow 0(U1)1 \rightarrow 0(\epsilon)11 = 011$.